



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Technischer Entwurf

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 22. Juni 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Lucas Neidahl ,Luan Hyseni, Marvin Voigt, Mohammed Alhalabi,	aus dem Fachentwurf (angepasst)
1.1	Luan Hyseni, Lucas Neidahl, Marvin Voigt	aus dem Fachentwurf (angepasst)
2	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.1	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.2	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.3	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.4	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.5	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.6	Lucas Neidahl	aus dem Fachentwurf (noch nicht korrigiert)
2.7	Mohammed Alhalabi	aus dem Fachentwurf (noch nicht korrigiert)
2.8	Mohammed Alhalabi	aus dem Fachentwurf (noch nicht korrigiert)
3	Lucas Neidahl, Marvin Voigt	Fertig
3.1	Lucas Neidahl, Marvin Voigt	angefangen /komponentendia und grobe beschreibung
3.2	Lucas Neidahl, Marvin Voigt	angefangen
3.3	Lucas Neidahl, Marvin Voigt	...
4	...	...
5	Luan Hyseni , Moha-med Sabri Jlizi	...

5.1	...	...
6	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
6.1	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
6.2	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
7	Marvin Voigt	aus dem Fachentwurf /aufgabenstel- lung noch vorhanden!!
7.1	Marvin Voigt	aus dem Fachentwurf
7.2	Marvin Voigt	aus dem Fachentwurf
7.3	Marvin Voigt	aus dem Fachentwurf
7.4	Marvin Voigt, Moham- med Alhalabi	aus dem Fachentwurf
7.5	Marvin Voigt	aus dem Fachentwurf
7.6	Marvin Voigt, Moham- med Alhalabi	aus dem Fachentwurf
8	Mohammed Alhalabi	...
9	Mohammed Alhalabi	...
9.1	Mohammed Alhalabi	...
9.2	Mohammed Alhalabi	...
9.3	Mohammed Alhalabi	...
10	Mohammed Alhalabi	...

Inhaltsverzeichnis

1	Einleitung	7
1.1	Projektübersicht	7
1.2	Projektdetails	8
1.2.1	Szenenkonfiguration	8
1.2.2	Drohne (Drone Lifecycle)	10
1.2.3	Aktivitätsdiagramm für den Workflow Bild zu Formation	11
2	Analyse der Produktfunktionen	12
2.1	Analyse von Funktion F10: Konfiguration der Show	12
2.2	Analyse von Funktion F11: Szene anlegen und Metadaten erfassen	13
2.3	Analyse von Funktion F12: Formation und Bewegung definieren	14
2.4	Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren	15
2.5	Analyse von Funktion F14: Szene speichern und zur Simulation übergeben	16
2.6	Analyse von Funktion F20: Choreographie simulieren	17
2.7	Analyse von Funktion F30: Projekt verwalten	19
2.8	Analyse von Funktion F40: Szenenprüfung auf Eingabefehler	20
3	Resultierende Softwarearchitektur	22
3.1	Komponentenspezifikation	22
3.2	Schnittstellenspezifikation	25
3.3	Komponentenspezifikation	26
3.4	Schnittstellenspezifikation	27
3.5	Protokolle für die Benutzung der Komponenten	27
4	Verteilungsentwurf	28
5	Implementierungsentwurf	29
5.1	Implementierung von Komponente C20 – ShowKonfigurator:	29
5.1.1	Paket-/Klassendiagramm	29
5.1.2	Erläuterung	30
6	Datenmodell	32
6.1	Diagramm	32
6.2	Erläuterung	33

7	Konfiguration	37
7.1	Technische Voraussetzungen	37
7.2	Projektstruktur	38
7.3	Start und Nutzung der Anwendung	38
7.4	Konfigurationsdateien	39
7.5	Konfiguration über die Benutzeroberfläche	40
7.6	Mögliche Erweiterungen	40
8	Änderungen gegenüber Fachentwurf	42
9	Erfüllung der Kriterien	43
9.1	Musskriterien	43
9.2	Sollkriterien	43
9.3	Kannkriterien	43
10	Glossar	44

Abbildungsverzeichnis

1.1	Ein Aktivitätsdiagramm für die Software	7
1.2	Zustandsdiagramm der Szenenkonfiguration	8
1.3	Zustandsdiagramm zum Drohnenlebenszyklus	10
1.4	Aktivitätsdiagramm der Bild zu Coordinate Funktion	11
2.1	Sequenzdiagramm für F10	13
2.2	Sequenzdiagramm für F11	14
2.3	Sequenzdiagramm für F12	15
2.4	Sequenzdiagramm für F13	16
2.5	Sequenzdiagramm für F14	17
2.6	Sequenzdiagramm für F20	18
2.7	Sequenzdiagramm für F30	20
2.8	Ein beispielhaftes Sequenzdiagramm	21
3.1	Komponentendiagramm	22
3.2	Ein beispielhaftes Komponentendiagramm.	26
4.1	Ein beispielhaftes Verteilungsdiagramm	28
5.1	Ein beispielhaftes Klassendiagramm für Komponente $\langle C10 \rangle$	30
6.1	Klassendiagramm Produktdaten	32

er ein neues oder ein altes Projekt geladen hat, kann er dieses nun bearbeiten, ein anderes Projekt auswählen oder eine Simulation starten. Wenn er ein Projekt bearbeiten will, kommt er in den Bearbeitungsmodus und kann verschiedene Parameter für die Simulation festlegen, wie etwa den Formations-Typ, den Hintergrund wählen, das Drohnenmodell auswählen, Bilder hochladen, die Anzahl der Drohnen festlegen und Lichteffekte definieren. Wenn dann alle nötigen Parameter festgelegt wurden, kann die Simulation gestartet werden. Der Nutzer kann auch die Simulation wieder abbrechen und das Projekt wieder bearbeiten oder ein anderes Projekt auswählen. Jederzeit kann das Programm auch beendet werden.

1.2 Projektdetails

1.2.1 Szenenkonfiguration

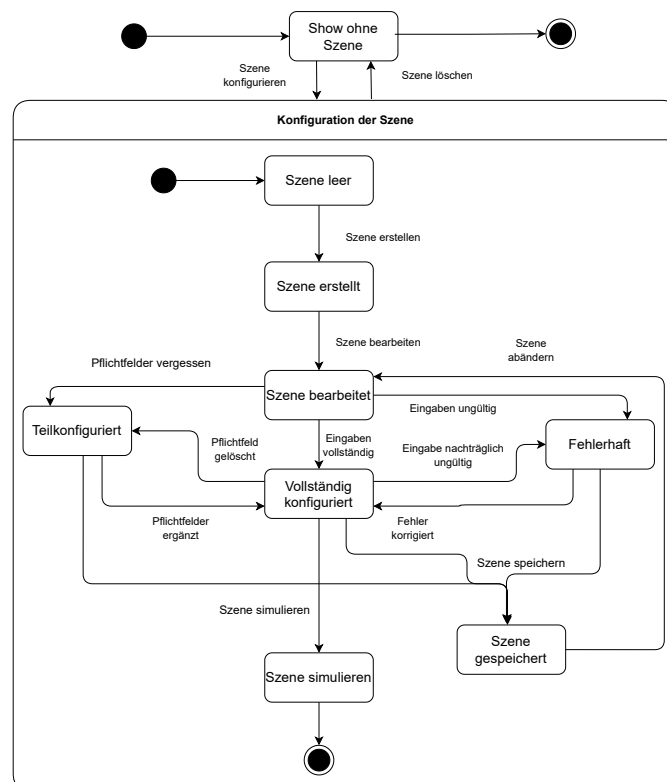


Abbildung 1.2: Zustandsdiagramm der Szenenkonfiguration

Das zweite Zustandsdiagramm stellt den internen Ablauf innerhalb der Szenenkonfiguration dar. Es beschreibt detailliert die möglichen Zustände einer Szene von der Erstellung bis zur Simulation. Der Prozess beginnt mit dem Zustand Szene leer, aus dem heraus eine neue Szene erstellt wird. Nach der Erstellung befindet sich die Szene im Zustand Szene erstellt und kann dann bearbeitet werden. Im Bearbeitungsprozess unterscheidet das System zwischen gültigen und ungültigen Eingaben. Ungültige Eingaben führen zum Zustand Fehlerhaft, während vollständige Eingaben den Zustand Vollständig konfiguriert erreichen. Eine Szene kann auch nur teilkonfiguriert sein, wenn beispielsweise Pflichtfelder fehlen oder gelöscht wurden. In diesem Fall kann sie nach entsprechender Korrektur erneut bearbeitet oder vervollständigt werden. Szenen können unabhängig vom Konfigurationsstand gespeichert und später weiterbearbeitet werden. Eine Simulation ist nur möglich, wenn alle relevanten Eingaben vollständig und korrekt vorliegen. Auch die Rückführung aus fehlerhaften Zuständen zur Bearbeitung ist explizit vorgesehen und modelliert. Beide Diagramme verdeutlichen, wie der SkyOrchestrator komplexe Abläufe strukturiert und intern abbildet. Sie dienen als Grundlage für die Umsetzung der Benutzerinteraktion sowie der automatischen Validierung innerhalb des Systems. .

1.2.2 Drohne (Drone Lifecycle)

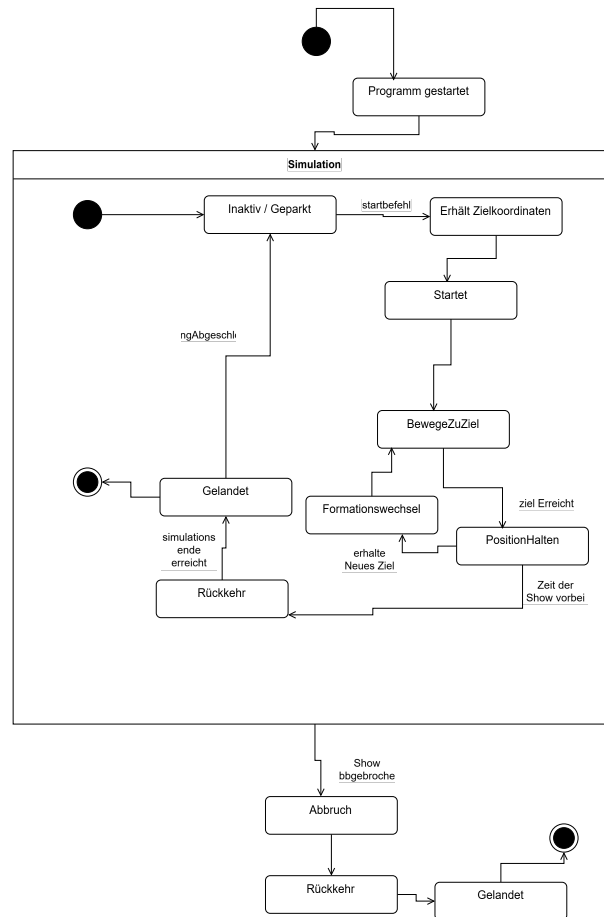


Abbildung 1.3: Zustandsdiagramm zum Drohnenlebenszyklus

Zu Beginn befindet sich die Drohne inaktiv/geparkt und wartet auf einen Startbefehl, woraufhin sie abhebt und auf eine Schwebhöhe steigt (Startet). Anschließend wechselt sie je nach erhaltenen Zielkoordinaten in den Zustand „BewegeZuZiel“ und verharrt nach Erreichen ihrer Position im Zustand „PositionHalten“. Bei einem Formationswechsel erhält sie neue Zielkoordinaten und wechselt in den Zustand „Formationswechsel“, bis auch hier die Position erreicht ist und sie zurück in „PositionHalten“ geht. Nach dem Simulationsende folgt der Zustand „Rückkehr“, in dem sie zur Startposition zurückkehrt, und schließlich „Gelandet“, sobald die Parkposition erreicht und die Landung abgeschlossen ist. Das Program bzw die Simulation kann auch jederzeit abgebrochen werden, in dem die Drohnen zurückkehren und wieder landen, von ihrer jetzigen Position.

1.2.3 Aktivitätsdiagramm für den Workflow Bild zu Formation

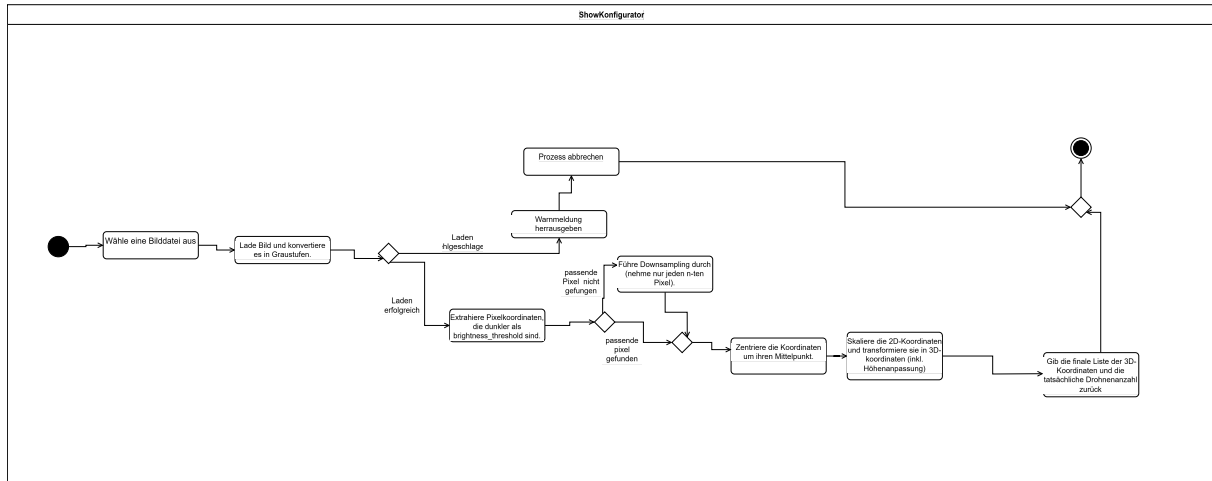


Abbildung 1.4: Aktivitätsdiagramm der Bild zu Coordinate Funktion

Der Workflow beginnt mit der Auswahl einer Bilddatei über einen Dateidialog. Anschließend wird das Bild geladen und in Graustufen konvertiert. Falls das Laden fehlschlägt, wird der Prozess abgebrochen. Andernfalls werden alle Pixel extrahiert, deren Helligkeit unter einem festgelegten Schwellenwert liegt. Wenn dabei keine passenden Pixel gefunden werden, endet der Prozess ebenfalls. Sind jedoch Pixel vorhanden, wird geprüft, ob deren Anzahl die Zielanzahl an Drohnen überschreitet. Ist das der Fall, erfolgt ein Downsampling, bei dem nur jeder n-te Pixel berücksichtigt wird. Danach werden die Koordinaten um ihren Mittelpunkt zentriert und in 3D-Koordinaten (x,y,z) transformiert, wobei auch die Höhe angepasst wird. Abschließend wird die finale Liste der 3D-Koordinaten zusammen mit der tatsächlichen Drohnenanzahl zurückgegeben, der Prozess ist damit abgeschlossen.

2 Analyse der Produktfunktionen

Dieses Kapitel analysiert die einzelnen Produktfunktionen aus dem Pflichtenheft, um die spätere Architekturentwicklung zu ermöglichen. Jede Funktion wird in einem eigenen Unterkapitel dokumentiert.

2.1 Analyse von Funktion F10: Konfiguration der Show

Diese Funktion ermöglicht dem Benutzer die Planung der Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen. Die Anforderungen RM1, RM2, RM4, RM6 und RM7 sind hierbei zentral. Beim Anlegen einer neuen Szene klickt der Nutzer in der GUI auf "Neue Szene erstellen". Daraufhin wird im Hintergrund der ShowKonfigurator aufgerufen, etwa über eine Methode wie `neueSzeneAnlegen()`. Diese Funktion ruft die ShowKonfigurator-Komponente auf, welche mit Hilfe des Projektmanagement-Moduls ein neues leeres Szenen-Objekt erstellt. Anschließend gibt das Projektmanagement Modul eine Bestätigung. Der Show-Konfigurator informiert daraufhin die GUI, welche die Anzeige aktualisiert und dem Nutzer die neu erstellte Szene und weitere Eingabefelder zeigt, um die Szene nun zu konfigurieren.

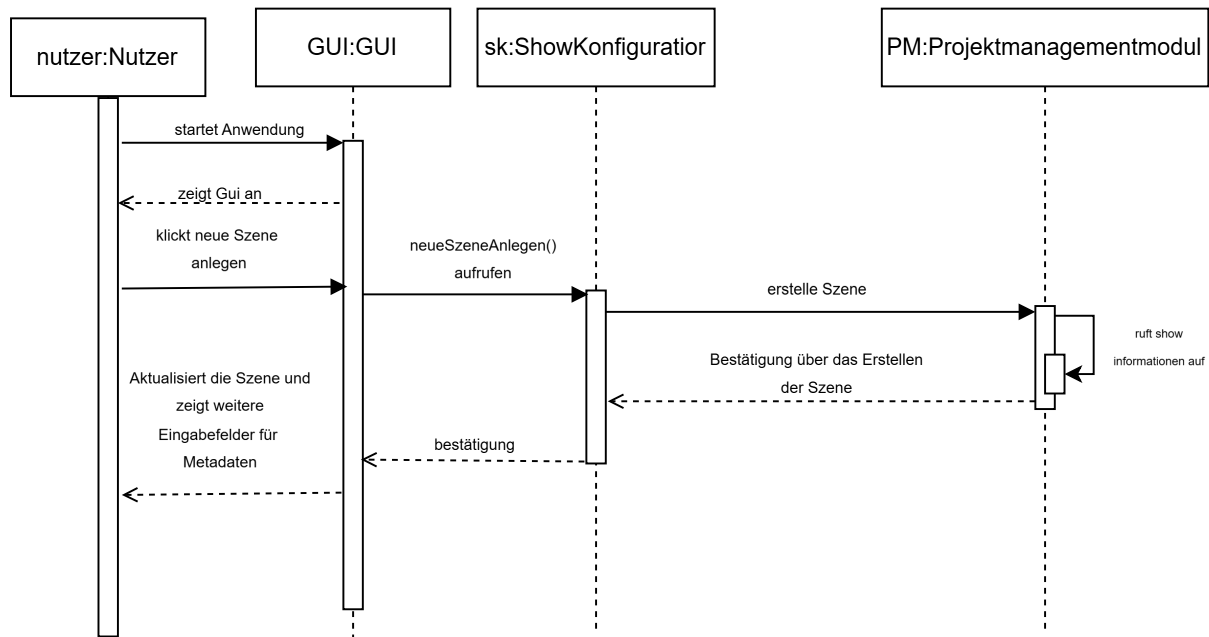


Abbildung 2.1: Sequenzdiagramm für F10

2.2 Analyse von Funktion F11: Szene anlegen und Metadaten erfassen

Beim Anlegen einer neuen Szene wird der Nutzer in der GUI aufgefordert, grundlegende Metadaten wie Name, Dauer und Drohnenanzahl einzugeben. Diese Metadaten werden anschließend vom GUI-Element an den ShowKonfigurator übergeben, der sie nutzt, um eine neue Szene zu initialisieren. Der ShowKonfigurator reicht die Metadaten weiter an das Projektmanagement-Modul, welches ein neues Szenenobjekt erstellt und die übergebenen Informationen speichert. Nach erfolgreicher Anlage gibt das Projektmanagement Modul eine Bestätigung oder das erstellte Objekt zurück, woraufhin die GUI entsprechend aktualisiert wird und dem Nutzer die leere Szene oder weitere Bearbeitungsmöglichkeiten anzeigt.

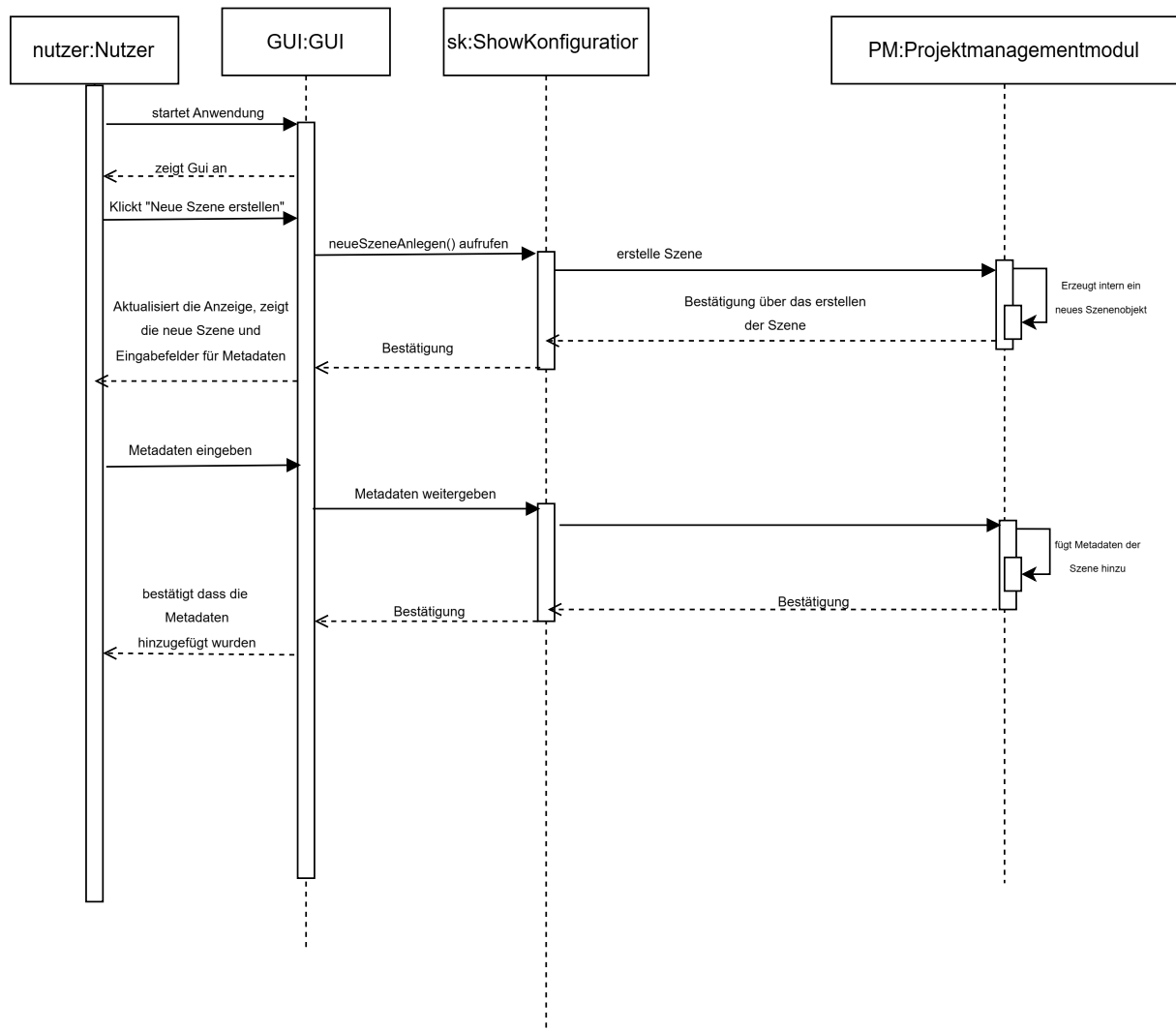


Abbildung 2.2: Sequenzdiagramm für F11

2.3 Analyse von Funktion F12: Formation und Bewegung definieren

Beim Auswählen einer Formation ist eine Szene bereits in der GUI geöffnet und aktiv, die Szenenverwaltung kennt die aktuelle Drohnenanzahl. Der Nutzer klickt auf „Formation auswählen“, woraufhin die GUI dem ShowKonfigurator die aktuelle Szene übermittelt. Der ShowKonfigurator fragt in der FormationsBibliothek nach verfügbaren Formationen und erhält eine Liste.. Diese Optionen werden der GUI angezeigt. Nachdem der Nutzer eine Formation ausgewählt hat, teilt die GUI dem ShowKonfigurator die gewählte Formation mit. Der ShowKonfigurator

holt über die SzenenVerwaltung die Szenendetails. Der ShowKonfigurator wendet die Formation an, indem er das Projektmanagement-Modul die Anwendung der Formation übergibt. Das Projektmanagement-Modul berechnet basierend auf Formation und Drohnenanzahl die exakten Positionen jeder Drohne und aktualisiert deren Zustände. Nach Bestätigung der Anwendung aktualisiert der ShowKonfigurator die GUI.

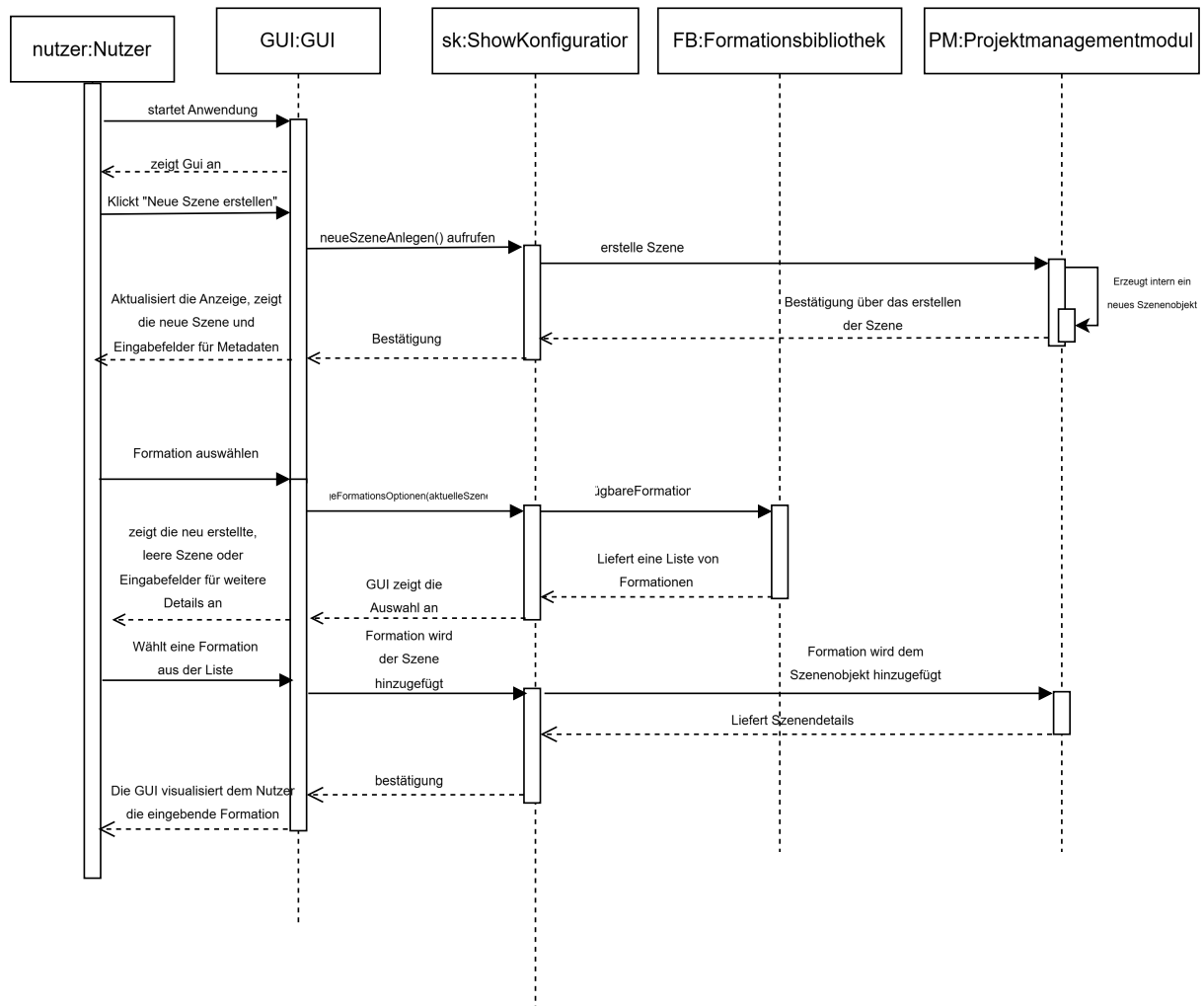


Abbildung 2.3: Sequenzdiagramm für F12

2.4 Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren

Beim Konfigurieren eines Lichteffekts ist zunächst eine Szene in der GUI geöffnet und enthält bereits Drohnen. Der Nutzer klickt auf „Lichteffekt hinzufügen“ oder wählt in der Timeline bzw. die gewünschten Drohnen aus. Die GUI übermittelt daraufhin an den ShowKonfigurator die

aktuelle Szene, die ausgewählten Drohnen und den Zeitpunkt in der Timeline. Der ShowKonfigurator holt über die LichtEffektBibliothek eine Liste verfügbarer Effekte und zeigt diese der GUI an. Nachdem der Nutzer einen Effekt (beispielsweise „Rot Dauerlicht“) samt optionaler Parameter wie Dauer und Helligkeit ausgewählt hat, übergibt die GUI die Daten zurück an den ShowKonfigurator. Anschließend aktualisiert der ShowKonfigurator die GUI, sodass die Timeline und gegebenenfalls eine Vorschau den neu angelegten Lichteffect anzeigen.

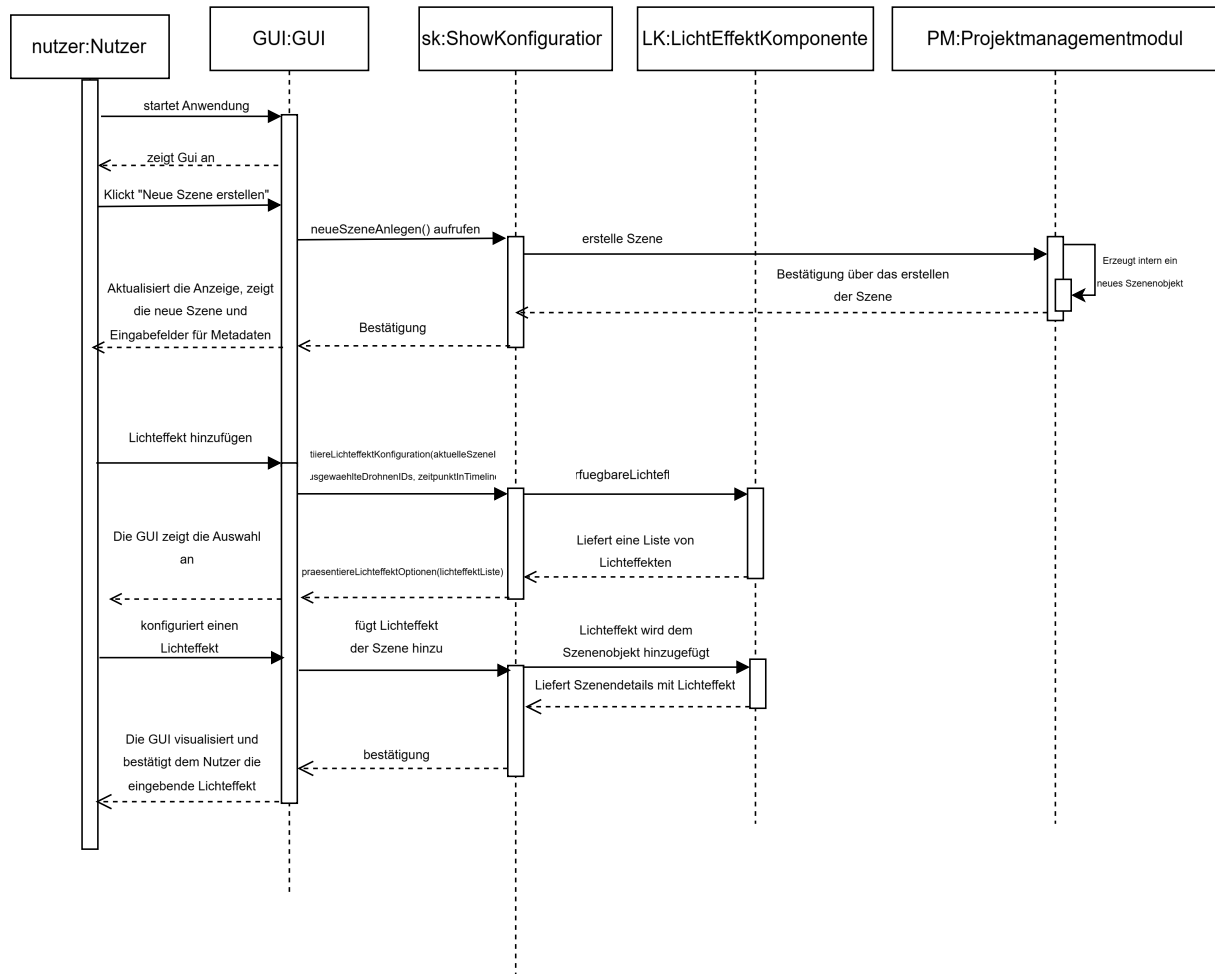


Abbildung 2.4: Sequenzdiagramm für F13

2.5 Analyse von Funktion F14: Szene speichern und zur Simulation übergeben

Die Simulation einer Szene beginnt, wenn der Nutzer in der geöffneten und vollständig eingerichteten Szene auf „Simulation starten“ klickt. Die GUI leitet diesen Befehl an den ShowKonfi-

gurator weiter, der daraufhin alle benötigten Szenendaten von dem Projektmanagement-Modul anfordert. Nachdem der ShowKonfigurator die vollständigen Informationen zurückerhalten hat, kann er die Szene, falls gewünscht, zunächst über die SpeicherKomponente sichern. Anschließend bereitet der ShowKonfigurator die Daten in einem simulationsgerechten Format auf und übergibt sie an die Simulationsbibliothek (Pybullet). Sobald die Simulationsbibliothek den Eingang der Szene bestätigt und die Simulation initialisiert hat, informiert der ShowKonfigurator die GUI darüber, dass die Simulation gestartet wurde. Die GUI wechselt daraufhin in den Simulationsmodus und zeigt dem Nutzer an, wie die Drohneninformationen, Bewegungsabläufe und Lichteffekte ablaufen.

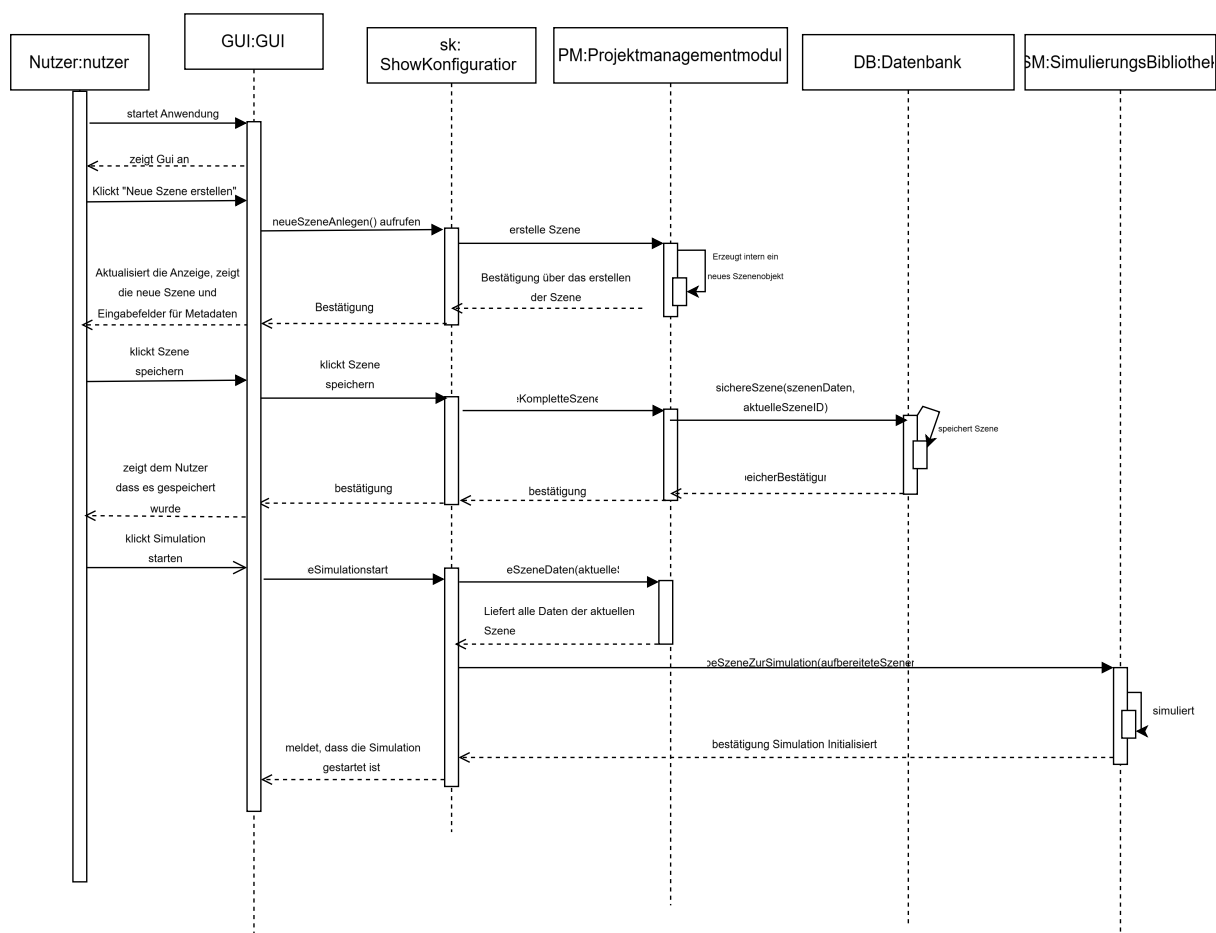


Abbildung 2.5: Sequenzdiagramm für F14

2.6 Analyse von Funktion F20: Choreographie simulieren

Beim Starten der Simulation lädt der Nutzer über die GUI-Modul die gewünschte Show (entweder die aktuell geladene oder eine gespeicherte) zur Anzeige und klickt auf „Show simulieren“.

Die GUI reicht den Befehl an das Konfigurationsmodul weiter, das alle relevanten Show-Daten vom Projektmanagement-Modul abrufen. Anschließend übergibt das Konfigurationsmodul diese vollständigen Daten an die Simulationsbibliothek. Dort wird die 3D-Umgebung initialisiert und die Drohnenobjekte entsprechend der ersten Szene positioniert. Sobald das Simulationsmodul die Daten erfolgreich übernommen hat, läuft die Simulation ab. Nachdem es die Simulation beendet hat, meldet es dem Konfigurationsmodul, dass die Show zu Ende ist. Dieses Modul leitet wiederum die Bestätigung an das Gui weiter. Das Gui wird wieder aktiv und der Nutzer kann erneut über das Programm die Show konfigurieren oder erneut simulieren.

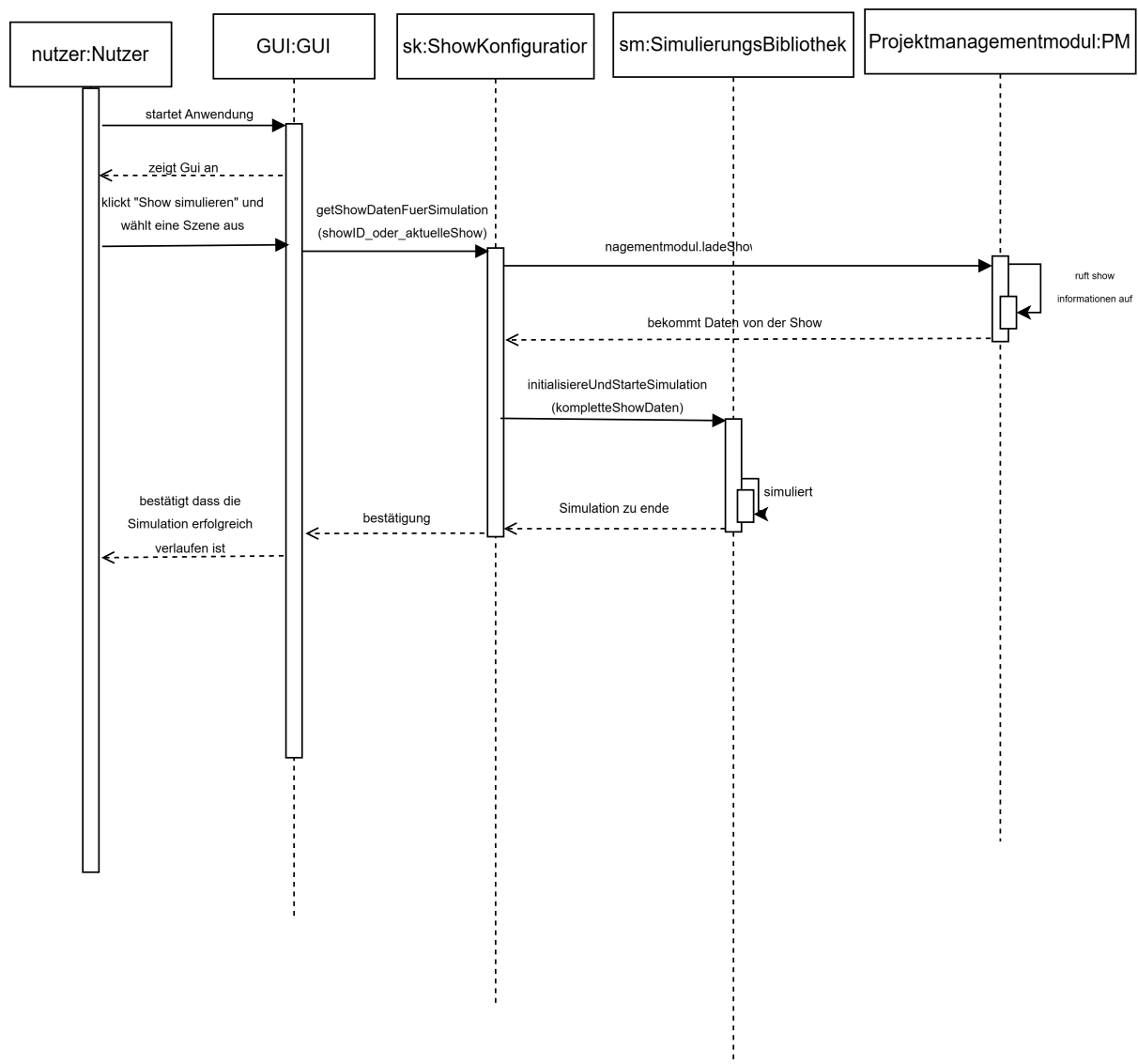


Abbildung 2.6: Sequenzdiagramm für F20

2.7 Analyse von Funktion F30: Projekt verwalten

Die Funktion F30: Projekt verwalten beschreibt das Speichern, Laden und Exportieren von Projekten innerhalb des SkyOrchestrator-Systems. Sobald der Benutzer in der grafischen Oberfläche eine entsprechende Aktion auslöst, beispielsweise über „Projekt speichern“ oder „Projekt laden“, leitet die GUI den Befehl an den ProjektManager weiter. Dieser übernimmt die Verarbeitung des Befehls, etwa durch Serialisierung der Projektdaten und Weitergabe an das Dateisystem im Falle eines Speichervorgangs oder durch Einlesen und Validierung bestehender Daten beim Laden eines Projekts. Der ProjektManager kommuniziert dabei direkt mit dem Dateisystem, ruft die jeweiligen Methoden auf und gibt nach erfolgreicher oder fehlerhafter Ausführung eine Rückmeldung an die GUI zurück. Diese aktualisiert daraufhin die Benutzeroberfläche und zeigt dem Benutzer das Ergebnis an. Auch Exportvorgänge werden über dieses Modul abgewickelt, indem Projektinformationen in ein geeignetes Austauschformat überführt und gespeichert werden. Die folgende Abbildung zeigt die beteiligten Komponenten sowie die Austauschbeziehungen im Rahmen eines typischen Speichervorgangs.

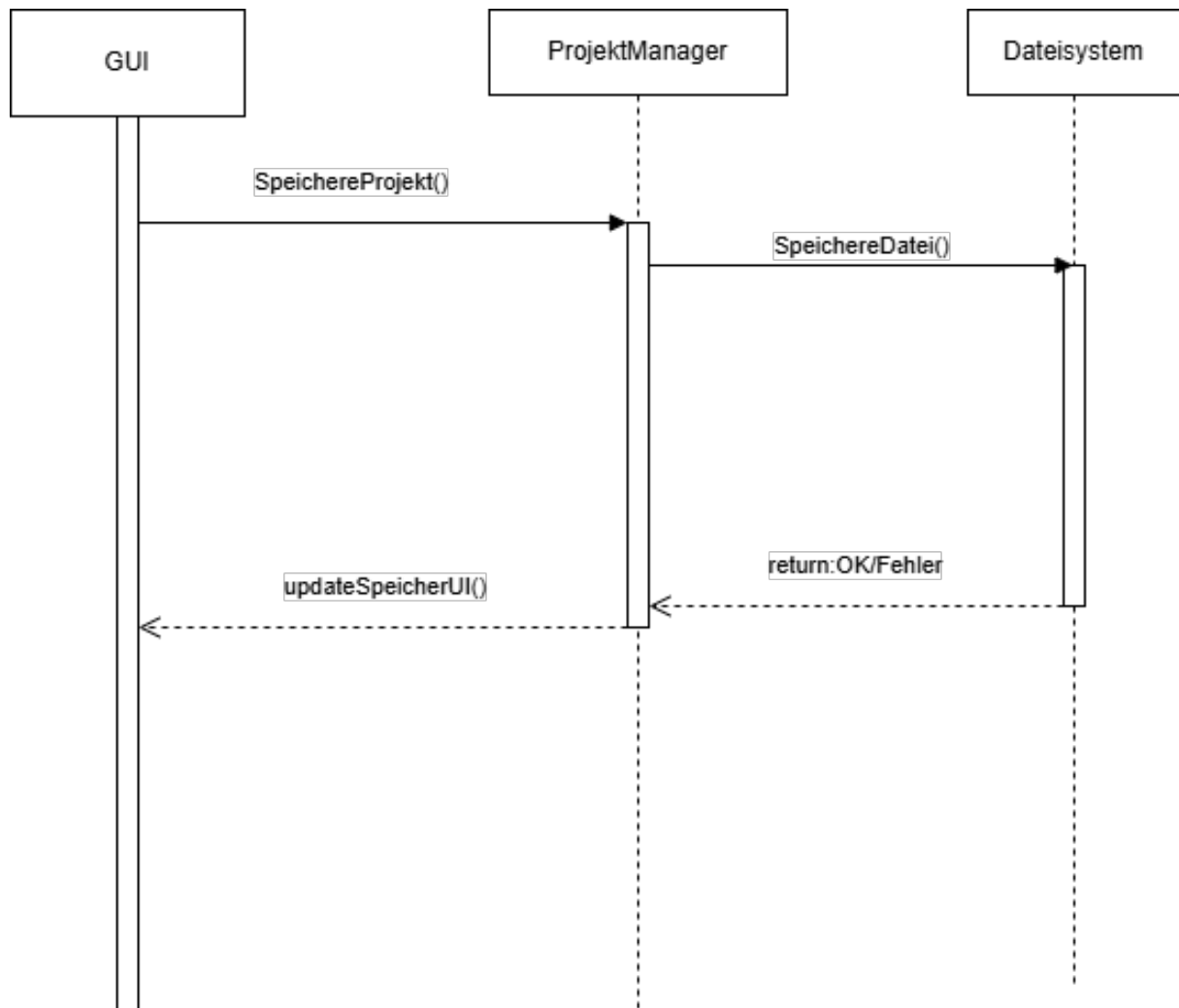


Abbildung 2.7: Sequenzdiagramm für F30

2.8 Analyse von Funktion F40: Szenenprüfung auf Eingabefehler

Die Funktion F40: Szenenprüfung auf Eingabefehler ermöglicht die automatisierte Überprüfung einer konfigurierten Szene auf Vollständigkeit und Plausibilität der eingegebenen Daten. Sobald der Benutzer über die grafische Benutzeroberfläche eine entsprechende Aktion auslöst, etwa durch Auswahl des Befehls „Szene prüfen“ oder durch den Versuch, eine Szene zu simulieren oder zu speichern, übermittelt die GUI eine Prüfanforderung an die Validierungskomponente. Diese greift auf die aktuellen Szenendaten der SzenenVerwaltung zu, analysiert sämtliche Parameter und Eingaben der Szene, und überprüft dabei insbesondere Pflichtfelder, Wertebereiche, zeitliche Abfolgen sowie geometrische Kollisionen oder doppelte Zuordnungen. Im Falle fehler-

hafter oder unvollständiger Daten generiert die Validierung eine strukturierte Fehlerliste und sendet diese an die GUI zurück. Die grafische Oberfläche visualisiert die Rückmeldung in geeigneter Form, beispielsweise durch Fehlermarkierungen oder Meldungsfenster, und ermöglicht dem Benutzer die gezielte Korrektur der betroffenen Elemente. Die folgende Abbildung zeigt den Nachrichtenaustausch zwischen den beteiligten Komponenten beim Ablauf einer Szenenprüfung.

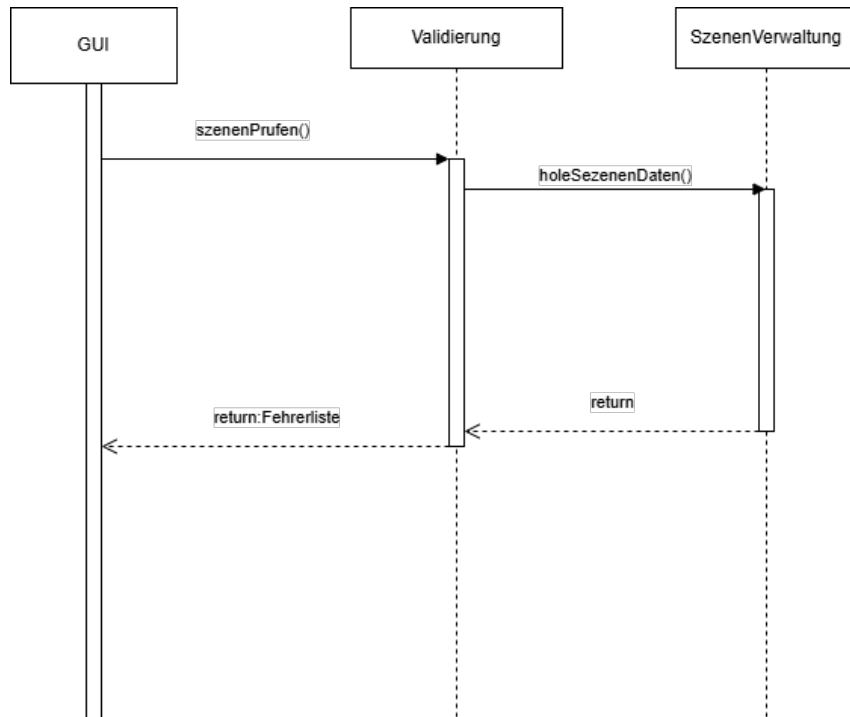


Abbildung 2.8: Ein beispielhaftes Sequenzdiagramm

Quelle der Abbildung 2.8: <http://www.uml-diagrams.org/>

3 Resultierende Softwarearchitektur

Dieses Kapitel beschreibt, basierend auf den analysierten Produktfunktionen aus Kapitel 2, die Softwarearchitektur von SkyOrchestrator. Die Architektur gliedert sich in mehrere Komponenten und Subsysteme, die jeweils klar definierte Aufgaben übernehmen.

3.1 Komponentenspezifikation

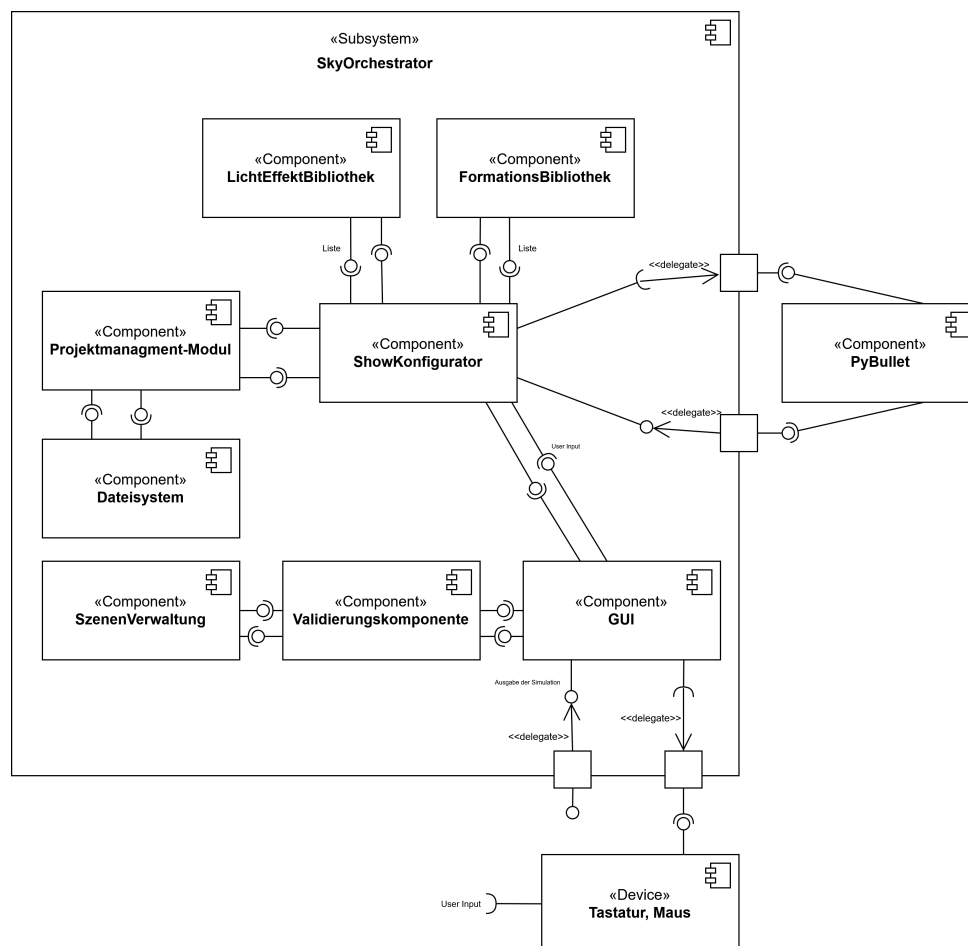


Abbildung 3.1: Komponentendiagramm

Komponente <C10>: <Graphical User Interface (GUI)>

Die GUI ist primär für die Interaktion mit dem Nutzer und die Visualisierung des Systemzustands verantwortlich. Sie stellt die gesamte Benutzeroberfläche für die Konfiguration von Szene und Show bereit. Ihre Hauptaufgabe umfassen die Entgegennahme und Verarbeitung von Nutzereingaben, um eine intuitive Bedienung zu gewährleisten. Des Weiteren ist die GUI für die Anzeigen von Rückmeldungen, Fehler und Simulationsergebnissen zuständig, wodurch der Nutzer stets über den aktuellen Systemstatus informiert wird. Die Kommunikation mit dem ShowKonfigurator erfolgt über definierte Schnittstellen, um die Steuerung der Anwendung zu ermöglichen und die Interaktion mit den weiteren Komponenten des Systems zu orchestrieren.

Komponente <C20>: <ShowKonfigurator>

Der ShowKonfigurator dient als zentrale Steuerungs- und Koordinationskomponente zwischen der GUI, dem Projektmanagement-Modul, der Formationsbibliothek, der Lichteffektbibliothek und der Simulationsbibliothek. Er verarbeitet die Nutzereingaben, verwaltet die aktuelle Szene sowie die zugehörigen Show-Daten und sorgt für die Anwendung der ausgewählten Formationen und Lichteffekte. Zudem übernimmt der ShowKonfigurator die Übergabe der konfigurierten Daten an das Projektmanagement-Modul zur weiteren Verarbeitung und stellt sicher, dass alle beteiligten Komponenten synchron zusammenarbeiten.

Komponente <C30>: <Projektmanagement-Modul>

Das Projektmanagement-Modul übernimmt die zentrale Verwaltung und Organisation der Projektdaten innerhalb des SkyOrchestrator-Systems. Es ist verantwortlich für die Berechnung der Drohnenpositionen basierend auf den ausgewählten Formationen und Bewegungsabläufen. Darüber hinaus sorgt das Modul für das Speichern und Laden von Projekten, um eine verlässliche Verwaltung der Show-Konfigurationen zu gewährleisten. Für die Simulation bereitet das Projektmanagement-Modul die Szenendaten auf und übergibt diese an die Simulationsbibliothek.

Komponente <C40>: <FormationsBibliothek>

Die FormationsBibliothek stellt eine Sammlung vordefinierter Drohnenformationen bereit, die vom ShowKonfigurator zur Konfiguration der Shows genutzt werden. Sie verwaltet die verfügbaren Formationstypen und ermöglicht deren Erweiterung, um auf unterschiedliche Anforderungen reagieren zu können.

Komponente <C50>: <LichtEffektBibliothek>

Die LichtEffektBibliothek stellt eine Sammlung vordefinierter Lichteffekte sowie deren zugehöriger Parameter bereit, die vom ShowKonfigurator zur Konfiguration der Shows verwendet werden. Sie verwaltet die verfügbaren Effekte und ermöglicht deren Erweiterung, um eine vielfältige Gestaltung von Lichtshows zu ermöglichen.

Komponente $\langle C60 \rangle$: $\langle \text{Simulationsbibliothek (PyBullet)} \rangle$

Aufgaben der Komponente

- Simulation der Drohnenchoreographie in einer 3D-Umgebung
- Initialisierung der Szene und Steuerung der Abläufe
- Rückmeldung des Simulationsstatus an den ShowKonfigurator

Komponente $\langle C70 \rangle$: $\langle \text{Validierungskomponente} \rangle$

Aufgaben der Komponente

- Prüfung der Szenendaten auf Vollständigkeit und Plausibilität
- Generierung von Fehlerlisten bei fehlerhaften Eingaben
- Kommunikation mit der GUI zur Visualisierung von Fehlern

Komponente $\langle C80 \rangle$: $\langle \text{DatenSysteme} \rangle$

Aufgaben der Komponente

- Verwaltung der Speicherung und des Ladens von Projektdaten (F14, F30)
- Bereitstellung von Schnittstellen zum Dateisystem für das Speichern, Laden und Exportieren von Projekten (F30)
- Unterstützung bei der Serialisierung und Validierung von Projektdaten (F30)
- Sicherstellung der Datenintegrität und Organisation der Datenformate
- Verwaltung der persistente Speicherung von Szenen und Show-Konfigurationen (F14)

Komponente $\langle C90 \rangle$: $\langle \text{SzenenVerwaltung} \rangle$

Aufgaben der Komponente

- Verwaltung und Organisation der Szenenobjekte und deren Metadaten (F11)
- Verwaltung der Zuordnung von Formationen, Bewegungen und Lichteffekten zu Szenen (F12, F13)
- Bereitstellung der aktuellen Szenendaten für andere Komponenten wie ShowKonfigurator und Projektmanagement (F12, F14)
- Unterstützung bei der Validierung und Plausibilitätsprüfung der Szenendaten (F40)
- Verwaltung von Szenenänderungen, Erstellung, Aktualisierung und Löschung von Szenen (F11)

- Schnittstellenbereitstellung zur Kommunikation mit GUI, ShowKonfigurator und Validierungskomponente

3.2 Schnittstellenspezifikation

Schnittstelle $\langle I10 \rangle$: $\langle \text{Graphical User Interface (GUI)} \rangle$

Operation	Beschreibung
$\langle \text{TODO: Funktion} \rangle$	Übermittelt die vom Nutzer gewählte Formation an den ShowKonfigurator
$\langle \text{TODO: Funktion} \rangle$	Übermittelt die ausgewählten Lichteffekte und Parameter
$\langle \text{startSimulation}() \rangle$	Signalisiert den Start der Simulation
$\langle \text{TODO: Funktion} \rangle$	Zeigt Validierungsfehler in der GUI an
$\langle \text{TODO: Funktion} \rangle$	Aktualisiert die Timeline mit neuen Effekten oder Bewegungen

Schnittstelle $\langle I20 \rangle$: $\langle \text{ShowKonfigurator} \rangle$

Operation	Beschreibung
$\langle \text{TODO: Funktion} \rangle$	Berechnet und aktualisiert Drohnenpositionen basierend auf Formation

—————-ab hier alles später löschen ist nur die aufgaben stellung—————

Dieser Abschnitt hat die Aufgabe, einen Überblick über die zu entwickelnden Komponenten und Subsysteme zu liefern.

3.3 Komponentenspezifikation

In diesem Abschnitt wird die aus der Analyse der Produktfunktionen (Kapitel 2) resultierende Komponentenstruktur durch ein Komponentendiagramm beschrieben. Die Bezeichnungen und Anzahl der Komponenten sollten natürlich konsistent zu denen aus Kapitel 2 sein. Gab es Änderungen, so sollen diese in Kapitel 9 beschrieben werden.

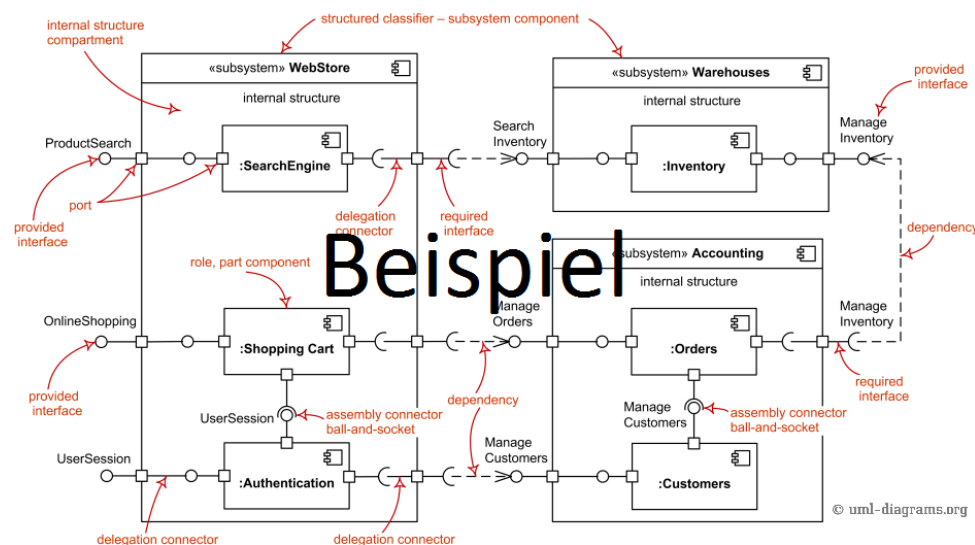


Abbildung 3.2: Ein beispielhaftes Komponentendiagramm.

Quelle der Abbildung 3.2: <http://www.uml-diagrams.org/>

Die einzelnen Komponenten werden anschließend kurz beschrieben.

Komponente $\langle C10 \rangle$: $\langle \text{Bezeichnung} \rangle$

Aufgaben der Komponente

Komponente $\langle C20 \rangle$: $\langle \text{Bezeichnung} \rangle$

Aufgaben der Komponente

3.4 Schnittstellenspezifikation

Im Folgenden werden die einzelnen Schnittstellen der Komponenten aus der Komponentenspezifikation näher erläutert, d.h. die von ihnen zur Verfügung gestellten Operationen werden dokumentiert. Die Tabelle ist dabei um so viele Zeilen zu erweitern, wie es Schnittstellen im Komponentendiagramm gibt. In der innen liegenden Aufteilung ist für jede Operation einer Schnittstelle eine Zeile einzufügen. Reine Set- und Get-Aufrufe brauchen nicht aufgeführt zu werden (sollten auch möglichst nicht komponentenübergreifend auftauchen).

Schnittstelle $\langle I10 \rangle$: **<Bezeichnung>**

Operation	Beschreibung
$\langle$ Signatur der Operation $\rangle$	$\langle$ Aufgabenbeschreibung der Operation $\rangle$

3.5 Protokolle für die Benutzung der Komponenten

In diesem Abschnitt wird mit Hilfe von Protokoll-Statecharts die korrekte Verwendung der zu entwickelnden Komponenten dokumentiert. Dies ist insbesondere für diejenigen Komponenten notwendig, für die eine Wiederverwendung möglich erscheint oder sogar bereits geplant ist.

Begründen Sie, für welche Komponenten eine Wiederverwendung sinnvoll erscheint und für welche nicht!

Fügen Sie so viele Statechartdiagramme ein, wie sie Komponenten gefunden haben.

4 Verteilungsentwurf

Sollte es sich bei dem Produkt um eine verteilte Anwendung handeln, so wird diese in diesem Abschnitt dokumentiert. Die Verteilung der Komponenten auf die beliebigen Knoten wird durch das folgende Verteilungsdiagramm beschrieben.

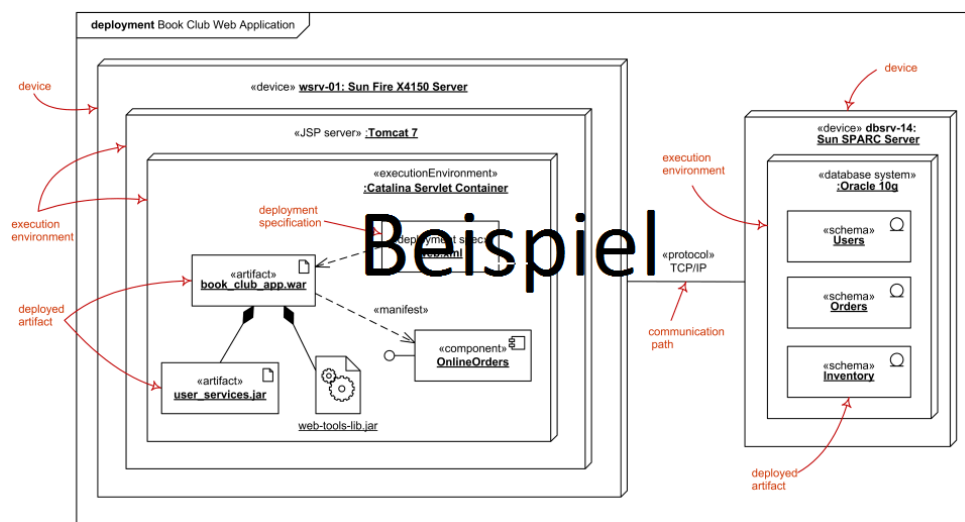


Abbildung 4.1: Ein beispielhaftes Verteilungsdiagramm

Quelle der Abbildung 4.1: <http://www.uml-diagrams.org/>

5 Implementierungsentwurf

In diesem Kapitel wird die technische Umsetzung der im Fachentwurf beschriebenen Systemarchitektur erläutert. Basierend auf der dort definierten Komponentenstruktur werden die zentralen Klassen und deren Beziehungen untereinander dargestellt. Ziel ist es, einen Überblick über die interne Struktur sowie die Interaktion der einzelnen Komponenten zu geben.

Für jede Hauptkomponente wird ein Klassendiagramm vorgestellt und beschrieben, wie deren Funktionalitäten durch Attribute und Methoden realisiert wurden. Dabei werden insbesondere die von außen zugänglichen Schnittstellen hervorgehoben, um die Zusammenarbeit der Module nachvollziehbar zu machen. Zusätzlich werden wichtige Abhängigkeiten zu externen Bibliotheken sowie genutzte Hilfsklassen aufgezeigt.

Die nachfolgenden Abschnitte widmen sich jeweils einer Komponente gemäß der Architekturübersicht aus Kapitel 3.

5.1 Implementierung von Komponente C20 – ShowKonfigurator:

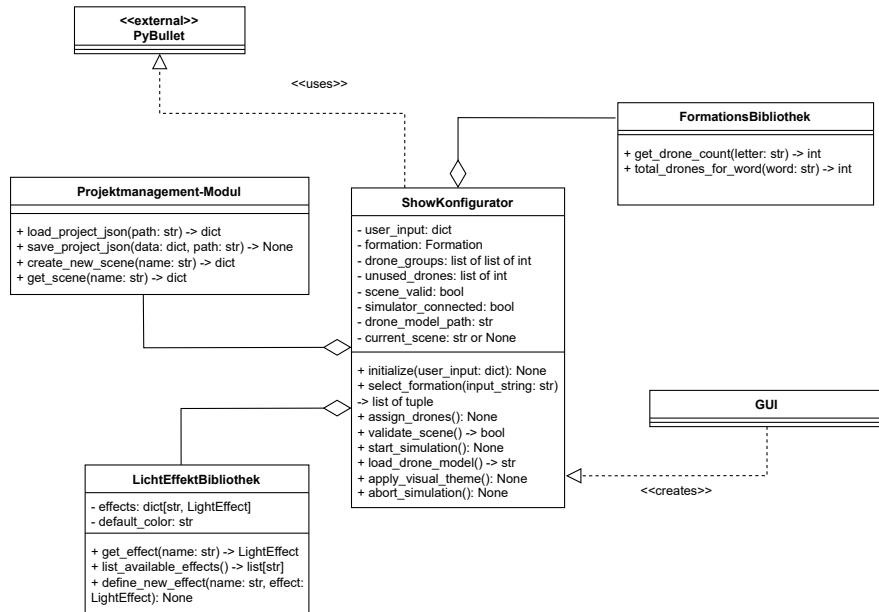
Die Komponente ShowKonfigurator ist das zentrale Modul zur Durchführung einer Drohnenshow. Sie übernimmt die Aufgabe, Nutzereingaben aus der GUI zu verarbeiten, Formationen zu laden, Drohnen zuzuweisen und die Simulation über PyBullet zu starten.

Technisch wird sie als eigenständige Python-Klasse realisiert, die mit anderen Modulen wie FormationsBibliothek, LichtEffektBibliothek und dem Projektmanagement interagiert. Die Simulationsschnittstelle erfolgt über Methodenaufrufe an die externe Bibliothek PyBullet.

5.1.1 Paket-/Klassendiagramm

In Abbildung 5.1 ist die Komponente ShowKonfigurator als Klassendiagramm dargestellt. Die Hauptklasse dieser Komponente ist ShowKonfigurator, welche die zentrale Steuerlogik für die Drohnenshow enthält.

Quelle der Abbildung 5.1: <http://www.uml-diagrams.org/>

Abbildung 5.1: Ein beispielhaftes Klassendiagramm für Komponente $\langle C10 \rangle$

5.1.2 Erläuterung

Attribute

- **input_data** – enthält alle Nutzereingaben zur Showkonfiguration
- **formation** – aktuelle Formation, z. B. ein Wort oder Symbol
- **drones_assigned** – Liste der Drohnen, die zugewiesen wurden

- `scene_valid` – boolescher Wert, ob Szene gültig ist

Methoden

- `initialize()` – Initialisiert Konfiguration, prüft Daten, lädt Module
- `assign_drones()` – Berechnet Drohnenverteilung gemäß Formation
- `start_simulation()` – Führt Simulation über PyBullet aus

Kommunikationspartner

- GUI: Übergibt Nutzereingaben («creates»)
- FormationsBibliothek: Berechnung Drohnenanzahl (Aggregation)
- LichtEffektBibliothek: Verwaltung visueller Effekte (Aggregation)
- Projektmanagement: Laden/Speichern von Szenen (Aggregation)
- PyBullet: Ausführung der Show («uses»)

6 Datenmodell

Die Anwendung SkyOrchestrator speichert die zentralen Informationen über geplante Shows, konfigurierte Drohnen, deren individuelle Flugbahnen, Formationen sowie durchgeführte Simulationen dauerhaft in einer dokumentenorientierten Datenbank. Die langfristig zu speichernden Daten werden in die Entitäten Show (E10), Drohne (E20), Flugbahn (E30), Formation (E40), Simulation (E50) und Wegpunkt (E60) aufgeteilt.

Ein Show-Objekt enthält Metadaten zur Drohnenshow und ist über eindeutige IDs mit den zugehörigen Drohnen, Formationen und Simulationen verknüpft. Jede Drohne besitzt eine eigene Flugbahn, die wiederum eine Liste von Wegpunkten beschreibt.

6.1 Diagramm

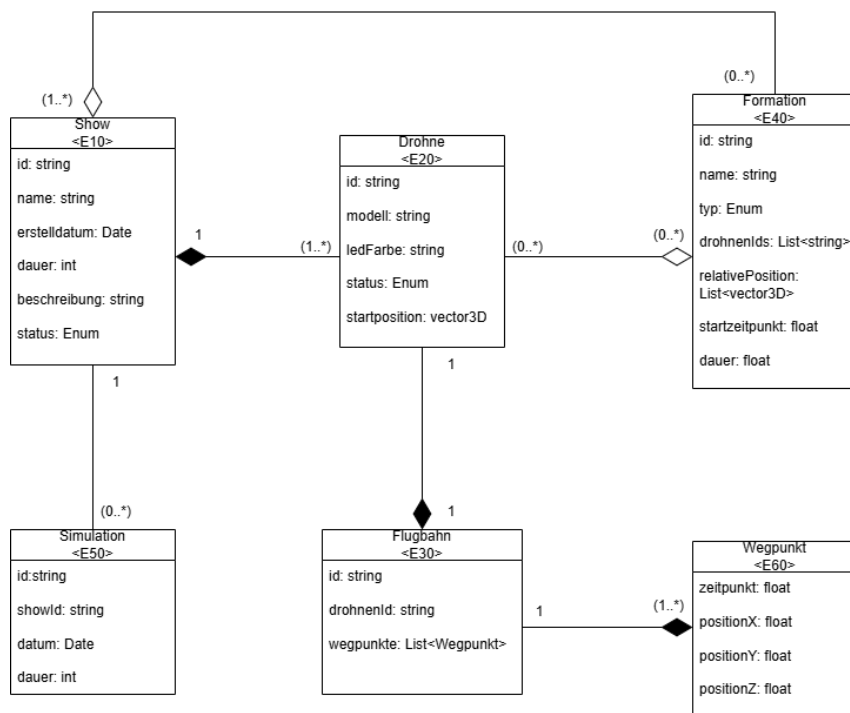


Abbildung 6.1: Klassendiagramm Produktdaten

6.2 Erläuterung

In den folgenden Tabellen wird die Beziehung zwischen den Entitäten Show $\langle E10 \rangle$, Drohne $\langle E20 \rangle$, Flugbahn $\langle E30 \rangle$, Formation $\langle E40 \rangle$, Simulation $\langle E50 \rangle$ und Wegpunkt $\langle E60 \rangle$ ausführlich dargestellt. Die Tabellen geben Aufschluss über die Art der Beziehung, die jeweilige Kardinalität, die erwartete Datenmenge sowie eine kurze inhaltliche Beschreibung. Grundlage der Modellierung sind die funktionalen Anforderungen an das System SkyOrchestrator.

Show $\langle E10 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 1–300 Drohnen je Show, pro Drohne ca. 1–2 KB	Einer Show können beliebig viele Drohnen zugeordnet werden. Jede Drohne gehört eindeutig zu einer Show.
Simulation	0..*	ca. 1–10 Simulationen, pro Show ca. 500–800 Byte	Eine Show kann mehrfach simuliert werden. Simulationen sind eindeutig einer Show zugeordnet.
Formation	0..*	ca. 0–100 Formationen, pro Show ca. 500–1000 Byte	Optional können Shows mehrere Formationen enthalten. Eine Formation beschreibt die gleichzeitige Anordnung mehrerer Drohnen.

Drohne $\langle E20 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	ca. 100–200 Byte	Jede Drohne ist genau einer Show zugeordnet.
Flugbahn	1	ca. 2–10 KB je Flugbahn	Jede Drohne besitzt genau eine Flugbahn, die als Liste von Wegpunkten gespeichert wird.
Formation	0..*	ca. 100–300 Byte Referenzdaten	Drohnen können in mehreren Formationen gleichzeitig oder nacheinander verwendet werden.

Flugbahn $\langle E30 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1	ca. 100–1000 Wegpunkte, gesamt ca. 2–10 KB	Die Flugbahn besteht aus einer Liste zeitlich geordneter Wegpunkte mit Informationen zu Position, Zeit, Bewegung und Lichtsteuerung.
Wegpunkt	1..*	ca. 100–1000 Wegpunkte	Eine Flugbahn besteht aus mehreren Wegpunkten, die den zeitlichen Verlauf bestimmen..

Formation $\langle E40 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 500–1000 Byte je Formation	Eine Formation umfasst mehrere Drohnen, deren Positionen relativ zur Formation angegeben werden.
Show	1..*	ca. 100–200 Byte	Jede Formation gehört eindeutig zu einer Show.

Simulation $\langle E50 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	ca. 500–800 Byte je Simulation	Eine Simulation ist eindeutig einer Show zugeordnet. Sie enthält u.a. Zeitstempel, Dauer, Kollisionserkennung und Energieverbrauch.

Wegpunkt $\langle E60 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Flugbahn	1	ca. 20–100 Byte je punkt	Eine Wegpunkt beschreibt eine konkrete Position der Drone zu einem bestimmten Zeitpunkt innerhalb ein Flugbahn z.b : Position (x, y, z), Zeit.

7 Konfiguration

In diesem Kapitel werden die technischen Voraussetzungen und Konfigurationsschritte beschrieben, die für die Installation, Ausführung und Nutzung von SkyOrchestrator erforderlich sind. Dazu zählen unter anderem die Systemvoraussetzungen, externe Abhängigkeiten sowie Hinweise zur Installation und zum Start der Anwendung.

7.1 Technische Voraussetzungen

Für die Ausführung von SkyOrchestrator wird eine lokale Python-Installation (Version 3.13 oder höher) sowie eine funktionierende Installation der Physik-Engine PyBullet benötigt, die für die dreidimensionale Darstellung und physikalische Simulation der Drohnenszenen verwendet wird. Darüber hinaus müssen weitere externe Python-Bibliotheken installiert werden, die in der mitgelieferten Datei `requirements.txt` aufgeführt sind. Diese befindet sich im Hauptverzeichnis von SkyOrchestrator und enthält folgende Abhängigkeiten:

- `numpy`
- `pybullet`
- `pandas`
- `scipy`

Diese Bibliotheken können über den folgenden Befehl im Terminal automatisch installiert werden:

```
python -m pip install -r requirements.txt
```

Hinweis: Im Verlauf der weiteren Entwicklung kann es dazu kommen, dass zusätzliche Bibliotheken benötigt werden. In diesem Fall wird die Datei `requirements.txt` entsprechend erweitert. Es empfiehlt sich daher, vor der Installation stets die aktuelle Version dieser Datei zu verwenden.

7.2 Projektstruktur

SkyOrchestrator ist modular aufgebaut und in verschiedene Verzeichnisse unterteilt, um die Wartbarkeit und Erweiterbarkeit der Software zu erleichtern. Im Folgenden wird ein Überblick über die wichtigsten Verzeichnisse und deren Funktionen gegeben:

- `main.py` – Startpunkt der Anwendung. Über diese Datei wird die grafische Benutzeroberfläche gestartet.
- `gui.py` – Quellcode der grafischen Benutzeroberfläche (GUI).
- `sim/` – Modul zur Durchführung der Simulationen mit PyBullet. Enthält die Funktionen zur Steuerung der Drohnenbewegungen.
- `models/` – Enthält die URDF Dateien der Dronen Modelle
- `savedProjects/` – Standardverzeichnis zur Speicherung von Benutzerszenen im `.json`-Format. Diese Dateien enthalten alle Konfigurationsdaten und können jederzeit erneut geladen werden.
- `requirements.txt` – Liste der externen Abhängigkeiten, die zur Ausführung des Programms erforderlich sind.
- `README.md` – Dokumentation mit Hinweisen zur Installation, Nutzung und zu häufig gestellten Fragen.

Diese Struktur stellt sicher, dass Benutzeroberfläche, Simulationslogik und Konfigurationsdaten klar voneinander getrennt sind. Dadurch wird nicht nur die Bedienung vereinfacht, sondern auch eine spätere Erweiterung des Systems erleichtert.

7.3 Start und Nutzung der Anwendung

SkyOrchestrator wird über die Datei `main.py` gestartet. Nach dem Wechsel in das Programmverzeichnis erfolgt der Start durch Eingabe des folgenden Befehls im Terminal:

```
python main.py
```

Nach dem Start öffnet sich das Hauptmenü der grafischen Benutzeroberfläche (GUI).

Der typische Arbeitsablauf im SkyOrchestrator beginnt mit dem Anlegen eines Projekts und einer oder mehrerer Szenen. Nach Eingabe der grundlegenden Metadaten (z.B. Name, Dauer,

Drohnenanzahl) werden Formationen, Bewegungen und Effekte konfiguriert. Die fertige Szene kann gespeichert, geladen, weiterbearbeitet und schließlich simuliert werden. Die einzelnen Schritte und Abläufe sind durch Zustands- und Sequenzdiagramme in den Kapiteln 1 und 2 detailliert beschrieben.

Die Simulation wird mit Hilfe von `PyBullet` dargestellt.

7.4 Konfigurationsdateien

SkyOrchestrator verwendet zur Speicherung und Wiederverwendung von Projekten sogenannte Sitzungsdateien im `.json`-Format. Diese Dateien werden standardmäßig im Unterordner `savedProjects` des SkyOrchestrator-Verzeichnisses abgelegt und können beim Speichern frei benannt werden. Der Zweck dieser Dateien besteht darin, sämtliche Einstellungen und Parameter einer Drohnenshow dauerhaft zu sichern und zu einem späteren Zeitpunkt wieder laden zu können.

Die in der `.json`-Dateien gespeicherten Parameter spiegeln die in Kapitel 2 beschriebenen Konfigurationsschritte wider. Dazu zählen insbesondere:

- **Metadaten der Szene:** Name, Gesamtdauer, Erstellungsdatum, Autor
- **Drohnenanzahl:** Anzahl der für die Szene vorgesehenen Drohnen
- **Formationsdaten:** Ausgewählte Formationen und deren zeitliche Abfolge
- **Bewegungsabläufe:** Zuordnung von Bewegungen zu einzelnen Drohnen oder Gruppen
- **Licht- und Zeiteffekte:** Art, Farbe, Zeitpunkt, Dauer und betroffene Drohnen
- **Timeline-Events:** Steuerung von Start, Ende, Formationswechseln und weiteren Effekten

Alle Parameter werden ausschließlich über die grafische Benutzeroberfläche eingestellt (siehe Abschnitt 4.4) und in der Datei gespeichert. Eine manuelle Bearbeitung der Konfigurationsdateien außerhalb der Anwendung ist nicht vorgesehen und wird nicht unterstützt.

Weitere Konfigurationsdateien, etwa zur Steuerung der Anwendung oder zur Systemkonfiguration, sind nicht erforderlich.

7.5 Konfiguration über die Benutzeroberfläche

SkyOrchestrator ermöglicht es dem Nutzer, sämtliche Einstellungen einer Drohnenshow komfortabel über die grafische Benutzeroberfläche vorzunehmen. Alle relevanten Steuerparameter – wie Drohnenanzahl, Formationen, Lichtanimationen, Zeitabläufe und weitere Szenenparameter – können individuell angepasst werden. Die zentrale Timeline-Komponente erlaubt es, Start- und Endzeitpunkte, Formationswechsel, Lichteffekte und Landungen präzise zu planen und zu steuern.

Während der laufenden Simulation bietet die Anwendung zusätzliche Funktionen, die die Interaktion und Kontrolle über den Ablauf erleichtern. Dazu zählen die freie Kamerasteuerung, mit der die Szene aus unterschiedlichen Perspektiven betrachtet werden kann, sowie die Möglichkeit, die Show über die Timeline zu verschiedenen Zeitpunkten zu analysieren. Außerdem kann die Simulation jederzeit vorzeitig abgebrochen werden, um Anpassungen vorzunehmen oder einen neuen Simulationsdurchlauf zu starten.

Alle grundlegenden Planungs- und Simulationsfunktionen sind so gestaltet, dass sie ohne Programmierkenntnisse bedient werden können. Eine Szene kann zu jedem Zeitpunkt gespeichert werden, unabhängig vom aktuellen Konfigurationsstand. Alle Eingaben werden vor der Simulation auf Gültigkeit geprüft. Die Anwendung prüft jedoch vor dem Start der Simulation, ob alle Pflichtfelder ausgefüllt und alle Eingaben gültig sind (siehe Zustandsdiagramm zur Szenenkonfiguration, Abbildung 1.3). Fehlerhafte oder unvollständige Szenen werden durch verständliche Hinweise kenntlich gemacht und können erst nach Korrektur simuliert werden.

7.6 Mögliche Erweiterungen

Zukünftige Versionen von SkyOrchestrator könnten um eine zentrale Konfigurationsdatei ergänzt werden, in der globale Parameter wie Drohnenanzahl, Startpositionen oder Lichtfarben dauerhaft gespeichert werden. Dies würde die Wiederverwendbarkeit und Portabilität erhöhen sowie die Trennung von grafischer Benutzeroberfläche und Konfigurationslogik verbessern.

Ebenfalls denkbar ist die Erweiterung um benutzerdefinierte Presets, die als Vorlagen für neue Shows dienen, sowie eine Versionsverwaltung für Projekte. Auch die Integration von Profileinstellungen, um verschiedene Nutzerpräferenzen zu unterstützen, wäre eine sinnvolle Erweiterung.

Diese Funktionen sind zum aktuellen Zeitpunkt noch nicht umgesetzt, werden jedoch als sinnvolle Ergänzungen für eine zukünftige Weiterentwicklung in Erwägung gezogen.

—— ab hier muss gelöscht werden ist nur die alte aufgaben stellung—— **Dieses Kapitel kann aus dem Fachentwurf übernommen werden, sollte jedoch die Bearbeitung der Annotationen beinhalten.**

Sollte für die Bearbeitung und Nutzung des Produktes ein Rechner/Server mit einer bestimmten Konfiguration erforderlich sein, so ist hier dessen Konfiguration zu beschreiben. Dies geschieht durch explizite Nennung aller Konfigurationsdateien und notwendiger Einträge.

Gibt es in eurem Projekt also config-Dateien so sollten diese hier alle aufgezählt werden. Es sollte beschrieben werden, wo diese zu finden sind und welchen Zweck sie erfüllen. Gibt es Parameter die angepasst werden können, so sollte auf diese eingegangen werden.

8 Änderungen gegenüber Fachentwurf

In diesem Kapitel sollt ihr Änderungen beschreiben, die gegenüber dem Fachentwurf geschehen sind. Es kann euch z.B. aufgefallen sein, dass ihr eine weitere Komponente benötigt, die ihr in Kapitel 2 nicht in den Sequenzdiagrammen beschrieben habt. Ihr solltet dazu dann die Sequenzdiagramme anpassen und hier erneut beschreiben. Gleiches gilt für die Kapitel Datenmodell und Konfiguration.

9 Erfüllung der Kriterien

Nachfolgend wird beschrieben, wie die einzelnen Kriterien des Pflichtenheftes erfüllt werden und worauf geachtet wird. Es ist dabei explizit auf die definierten Kriterien des Pflichtenheftes zu verweisen. Das Kapitel sollte fortlaufend gepflegt werden. **Es sollte auch darauf verwiesen werden, welche Komponente das Kriterium erfüllt.**

9.1 Musskriterien

Die folgenden Kriterien sind unabdingbar und müssen durch das Produkt erfüllt werden:

RM1 wird durch folgende Komponente umgesetzt.

RM2 ...

9.2 Sollkriterien

Die Erfüllung folgender Kriterien für das abzugebende Produkt wird angestrebt:

RS1 ...

RS2 ...

9.3 Kannkriterien

Die Erfüllung folgender Kriterien für das abzugebende Produkt wird angestrebt:

RC1 ...

RC2 ...

10 Glossar

Hier werden Fachbegriffe erklärt.